



Cartograph

Project Charter — natural-language querying of complex data warehouses with schema-graph retrieval and explainable join-path selection.

OWNER	Jagath Manjunath
DOCUMENT	Project Charter (deliverable of SDLC Phase 1)
VERSION	3 — amended 7 September 2026
ISSUED	5 September 2026
BUILD TARGET	15 September 2026 — with the first week of October held as contingency, not as plan
PHASE STATUS	Closed. All sections signed off; amended twice — on the definition of done, then after a capacity audit and a verification pass

What this document is for. This charter fixes every settled decision in one place, so that later phases build on a written foundation rather than a remembered one. Version 3 records the outcome of a capacity audit and a verification pass against the TPC-DS specification: the schedule gains a contingency window, the foreign-key risk is confirmed and largely retired, and two estimation errors are corrected on the record. Section 11 records every change.

01 Summary

A nine-day solo build of a working system, judged on whether it can explain its own reasoning — not merely produce an answer.

DELIVERABLE	WAREHOUSE	TEAM	CASH COST
Working web app + trace	TPC-DS on Postgres	Solo	LLM API only

Cartograph lets a non-technical user ask a business question in English against a large data warehouse and receive both the answer and a full account of how that answer was reached: which tables were retrieved, which join path was chosen, and which equally valid paths were not.

The project exists because the interesting failure in natural-language querying is not the query that errors — it is the query that succeeds while silently answering a different question than the one asked. Cartograph's position is that this class of failure cannot be eliminated, so it must be surfaced.

02 Purpose

This is a portfolio project first; the academic requirement is satisfied by the same artifact.

The project serves three audiences at once, and each judges it by a different standard. The definition of done in Section 9 is written to satisfy all three.

AUDIENCE	WHAT THEY ARE JUDGING	WHAT MUST THEREFORE EXIST
Academic examiner	Whether the work is understood, not merely produced	Visible reasoning, documented design decisions, defensible choices
Recruiter / interviewer	Whether the claim is specific and interrogable	One measured result, not an assertion of quality
Public viewer (demo video on X / LinkedIn)	Whether it is immediately legible in ninety seconds	The schema graph rendering with the chosen path highlighted

The project also carries a longer life than the build window: it stays on GitHub as a permanent public artifact.

03 Problem and thesis

Large language models fail on real warehouses through grounding, not syntax — and they fail quietly.

The situation

A mid-sized company holds its business history across dozens of interlinked warehouse tables. The people who need answers — managers and analysts — know exactly what they want to ask but do not know SQL, which table holds what, or how the tables join. They raise a ticket and wait days. The data team becomes a bottleneck and the organisation asks fewer questions than it should.

Why an LLM alone does not close the gap

Point a model at a small database and it writes correct SQL fluently. Point it at a real warehouse and it begins to fail — not on syntax, but on grounding. Given forty tables and three hundred columns and a question about "revenue in the western region," it must determine which of several columns containing the word *region* is meant, and which chain of joins connects a sale to a region at all.

THE FAILURE THAT MATTERS

In a retail warehouse there are typically **two valid paths from a sale to a region**: through the store where the sale occurred, and through the customer's address. Both are real foreign-key paths. Both are structurally correct. For a business selling both in-store and online they produce materially different answers.

The model picks one. The query runs. A clean, well-formatted table of figures comes back. **There is no error, and nothing indicates which interpretation was taken.** A decision-maker reads a confident number with no way to know it answered a different question. That is a silent failure — worse than a loud one, because it is wrong *and* hidden.

The response

First, stop showing the model the whole schema. A database already declares its own structure — tables, columns, primary keys, foreign keys. Read that metadata and build a graph: tables and columns are nodes, key relationships are edges. No inference is involved; it is reading what is already there. When a question arrives, find the few schema pieces that look relevant, walk the graph to find the join paths that actually connect them, and hand the model a small, relevant slice instead of a forty-table dump it must guess its way through. Less to get wrong.

Second, make the reasoning visible. Every answer returns with a trace: which tables and columns were retrieved, which join path was chosen, what other paths existed, and a plain-English account of how the query was assembled. When two routes to "region" exist, the user sees both and sees which was taken — and someone with domain knowledge can say *no, I meant the other one*, before acting on the number.

THESIS

Reduce the errors, and expose the ones that remain.

04 Scope

Everything specified in the two founding documents is in. Nothing beyond them is in.

The boundary is deliberately drawn against *new ideas arriving mid-build*, not against work already committed to. The project description and the stack sheet together define the full scope; no component named in either was cut, and no component absent from both is admitted without an explicit change to this charter.

IN SCOPE — THESIS CORE

- Read a live Postgres schema into a graph: tables and columns as nodes, foreign keys as edges
- Retrieve a relevant subgraph per question rather than dumping the full schema
- Find join paths on that graph, including detecting when more than one valid path exists
- Generate SQL, then validate it before execution — read-only, real columns only
- Return the answer together with the trace, and render the graph with the chosen path highlighted

IN SCOPE — SUPPORTING

- Application store for users, query history and traces
- Caching and rate limiting
- Database migrations
- Containerised local run; automated tests on push; public deployment

OUT OF SCOPE

Anything not named in the project description or the stack sheet, including in particular:

- Warehouse engines other than Postgres — MySQL, Snowflake, BigQuery dialects
- Multi-tenant or multi-database operation
- Conversational follow-up questions
- Charting or visualisation of result data
- Letting the user correct the join path and re-run
- Multi-schema-size degradation study (see §7, D-02)

CHANGE CONTROL

Any addition to scope requires an explicit amendment to this charter. This is the mechanism that protects the nine-day window, not willpower.

05 Feasibility

Assessed and cleared by the owner across all three dimensions.

DIMENSION	VERDICT	BASIS
Technical	Cleared	Every component of the stack is a mature, documented library performing a well-understood job. The one genuinely open problem is retrieval quality (R-02).
Resource	Cleared	Only the LLM API costs money; a few hundred rupees covers the build. All other tooling is free, and hosting fits a free tier.
Operational	Cleared	Solo delivery within a self-set nine-day window, with all academic requirements already satisfied and no external dependencies.

06 Constraints

The fixed walls. Every later decision must fit inside them, which means half the design questions answer themselves.

#	TYPE	CONSTRAINT
C-01	Technical	The stack sheet is the entire toolchain. No substitutions mid-build.
C-02	Technical	Postgres only. No other warehouse engine or SQL dialect in v1.
C-03	Environment	Development on a Windows laptop running WSL2 + Ubuntu. Everything must run there.
C-04	Deployment	Free-tier hosting. The deployed system must fit inside free-tier compute, memory and storage limits.
C-05	Resource	Solo developer. No parallelism, no delegation, no review partner.
C-06	Financial	LLM API is the sole paid component, budgeted at a few hundred rupees for the whole build.
C-07	Schedule	Build complete by end of 14 / early 15 September 2026. Self-imposed; the academic deadline is later.
C-08	Capability	Skill is being acquired during the project rather than before it, so build velocity is coupled to learning velocity.

07 Decisions taken

Each decision is recorded with the option rejected, so the reasoning survives the build.

#	DECISION	CHOSEN	REJECTED	RATIONALE
D-01	Demo warehouse	TPC-DS, generated via the DuckDB <code>tpcds</code> extension, exported to CSV and loaded into Postgres	Hand-built schema; Pagila as primary; official TPC toolkit	Hand-building was never necessary — standard warehouses exist. TPC-DS is the industry benchmark for exactly this workload, is genuinely too large to hold in one's head (which is what motivates the retrieval layer honestly), and contains the store-vs-customer-address ambiguity the thesis depends on. The DuckDB route avoids compiling the C toolkit, reducing setup to roughly an hour.
D-01a	Warehouse fallback	Pagila, the standard PostgreSQL sample database	—	Retained as fallback if the TPC-DS load stalls. Loads in one command and already contains an equivalent two-path ambiguity (a rental ties to both a store address and a customer address). Its weakness is size: at roughly fifteen tables the full schema fits comfortably in a model's context, which weakens the case for retrieval. Adequate to keep the build moving; inadequate as the headline demonstration.

#	DECISION	CHOSEN	REJECTED	RATIONALE
D-02	Evaluation design	One fixed set of ~15 questions with hand-written correct tables and join paths, run under two conditions: graph retrieval vs. full-schema prompting	Multi-schema-size degradation study across three warehouse sizes with decoy tables	The question set is required regardless as the only way to measure retrieval rather than eyeball it. Running it twice costs roughly an hour more and yields one measured, interrogable sentence — which is what the recruiter audience needs. The full study is a research project and does not fit the window.
D-03	Scope posture	Retain the full documented scope; cut nothing	Trim supporting components to protect the core	Owner's decision, made deliberately. Accepted consequence: some supporting components may land working-but-thin rather than polished. Preferred over arriving at the deadline with an incomplete system.
D-09	Schedule shape	Single push to 15 September, with the first week of October held as contingency	Planning the work across two windows from the outset; extending the sprint to 16-hour days	A capacity audit put the original eight-day plan at roughly 116 estimated hours against 96 available — about 20% over before anything went wrong. One day was added and October reserved. October is a backstop the owner intends not to use, not a second phase: the target remains 15 September. Sixteen-hour days were considered and rejected on estimation grounds — later hours produce defects that cost more time than they save.
D-10	Source of the foreign keys	The published tpeds_ri.sql from the TPC-DS toolkit, loaded into the overlay <i>and</i> attempted as database constraints	Transcribing relationships from the specification by hand; deriving them from the _sk naming convention	Verified 7 September: the toolkit already publishes roughly eighty ALTER TABLE ADD CONSTRAINT statements in dialect-agnostic SQL. Hand transcription was hours of error-prone work; derivation from naming was unsound — about half the columns break the convention (ss_addr_sk → customer_address, ss_demo_sk → customer_demographics, ss_sold_date_sk → date_dim) and a script built on it would have produced a confidently wrong graph. Loading into the overlay guarantees a correct graph regardless of whether the constraints apply; attempting them as constraints additionally preserves the project's founding claim that the graph is read from what the database itself declares.

#	DECISION	CHOSEN	REJECTED	RATIONALE
D-11	Contingency shape	Overrun slips into October rather than amputating a Core item	Deciding in advance which of authentication or deployment to sacrifice	With October reserved, the choice between two Core criteria disappears. If the sprint runs over, the tail moves rather than being cut — and the project is then finished and deployed in the first week of October, which is when applications open, and finished calmly rather than shipped in a panic.
D-05	API interaction model	Synchronous — one request, one response, connection held for the duration	Polling; server-sent progress streaming	Dramatically simpler at one concurrent user, and twenty seconds sits inside ordinary timeouts. Stage indication is driven client-side from known stage ordering. Accepted cost: a query that outruns the connection loses its answer even though it ran.
D-06	Join-path selection rule	Shortest path, ties broken deterministically, rule documented	Cardinality scoring; heuristics favouring fact tables	Cheaper, and truer to the thesis. The smarter the selector appears, the less a user feels obliged to check the trace — which is the behaviour the project exists to encourage. Confirmed in design.
D-07	Authentication depth	Full accounts — registration, login, hashed passwords, expiring sessions	Single seeded user with identity in the data model (initially recommended)	Reversal, by owner decision. The seeded-user option satisfied every mapped requirement at a fraction of the effort, and the recommendation was to defer auth to future work. Owner chose the full build for holistic completeness and real-world fidelity, accepting roughly 8–14 additional hours. Consequence: authentication enters the definition of done (criterion 12) rather than remaining an intention.
D-08	Requirements added during analysis	Seven additions beyond the founding documents, later joined by four more from the holistic audit	Holding strictly to the two source documents	Path conformance and plain-English path descriptions are load-bearing — without them the trace can be wrong or unreadable. The remainder (column candidates, unanswerable questions, foreign-key overlay, example questions, spend ceiling, layer tests, smoke check, frozen evaluation set, single API client) are cheap and justified. Recorded here because scope grew twice by explicit amendment, and that should be visible rather than invisible.
D-04	Delivery method	Scrum, adapted — sprints bounded by completed deliverables rather than fixed day counts	Hour-by-hour scheduling	The owner works to outcomes, not clocks. Detailed sprint structure is deferred until after Phase 3 (design), when the work is decomposable.

08 Risks

Two of the eleven remain live. R-09 was confirmed and retired the same day; R-10 is new and already mitigated by design.

#	RISK	WATCH	MITIGATION
R-09	Foreign keys are not declared — TPC-DS ships without primary or foreign key constraints, so an unmodified load produces a graph with no edges and the core premise collapses	Low was High	Confirmed and largely retired 7 September. Verified against two independent sources: 24 tables, no constraints of any kind in the DDL. But the toolkit publishes <code>tpcds_ri.sql</code> — roughly eighty ready-made foreign key statements — so the relationships never have to be authored. Loaded into the overlay, the graph is correct whether or not the database accepts them. The residual danger that a hand-written edge would be silently wrong is removed with the hand-writing.
R-10	Constraints may refuse to apply — generated TPC-DS data is known to contain rows whose keys do not resolve, so the foreign key statements can fail against a loaded warehouse	Low	Fully mitigated by design rather than by hope: the relationships are loaded into the overlay first and the graph is built from there. Applying them to the database is attempted second and is optional to the mechanism. Primary keys must be added to the seventeen dimension tables before any foreign key will apply.
R-01	TPC-DS load stalls — generation or CSV import into Postgres proves fiddlier than expected, blocking everything downstream	High	Timebox it. If the DuckDB route resists, fall back to Pagila (D-01a) immediately and revisit TPC-DS later. Nothing else can be tested until a warehouse exists, so this is the first thing to stand up.
R-02	Retrieval quality — selecting the correct subgraph from a plain-English question is the one component with no guaranteed recipe. Name-matching alone is a weak signal, and mediocre retrieval makes the whole demonstration mediocre	High	Author the evaluation question set early and tune against it rather than against impressions. This is the project's genuine engineering problem and deserves the largest share of attention.
R-03	Environment rabbit hole — WSL2, Docker, Postgres, pgvector and Node are all first-time installs; days can	Medium	Stand every layer up end-to-end with a trivial hello-world before building any feature. A stack that cannot stand up empty will not stand up full.

#	RISK	WATCH	MITIGATION
	disappear for reasons unrelated to the project		
R-04	Peripheral work crowds out the core — auth, caching, migrations, CI and deployment are all in scope and will happily consume days that belong to the thesis	Medium	Sequence deliberately: the five core capabilities first, supporting components after. Sequencing is set during planning, after design.
R-05	Frontend graph is the centrepiece and the least familiar work — the highlighted join path is what the public audience actually sees, and React is new	Medium	Do not leave it until last. Treat it as a core deliverable, not presentation polish.
R-06	LLM cost and latency — a retry loop can consume the budget quickly, and slow answers undermine the demo	Low	Cap retries, log every call with its cost, cache repeated questions. Redis is already in the stack for this purpose.
R-07	Single point of failure — illness, hardware failure, connectivity	Low	Push to GitHub continuously. Nothing lives only on the laptop.
R-08	Schema size undercuts the premise — on a small warehouse the retrieval layer is not obviously necessary, and a sharp examiner will say so	Low	Addressed structurally by D-01 (TPC-DS over Pagila) and evidentially by D-02 (the two-condition comparison produces a number rather than a claim).

09 Definition of done

Fifteen criteria, each answerable yes or no, derived from the three audiences in Section 2.

The project is done when every statement below is true and demonstrable. Each is written to be answerable yes or no, so that "done" cannot quietly move. Criteria 12 to 15 were added in version 2; criterion 1 to 11 are unchanged from the signed-off original.

WHY VERSION 2 EXTENDED THIS SECTION

The original criterion 2 required a trace to **exist**. It did not require the trace to be **true** — that the SQL which actually ran joined along the path the trace reports — nor to be **readable** by the non-technical person it is written for. Those are the project's two central properties, and the definition of done was satisfiable without either. Criteria 13 and 14 close that gap. Criterion 12 makes the authentication commitment accountable rather than merely intended, and criterion 15 covers the one catastrophic failure a public repository can suffer.

#	SERVES	CRITERION
1	All	A user can type a plain-English question against the TPC-DS warehouse and receive a correct result table.
2	Examiner	Every answer is accompanied by a trace showing the tables and columns retrieved, the join path chosen, and the alternative paths that existed.
3	Public	The schema graph renders on screen with the chosen join path visibly highlighted.
4	Examiner	Any generated SQL that is not a read-only SELECT is rejected before execution, and this can be demonstrated live.
5	Examiner	At least one demo question provably surfaces two valid join paths and reports which was taken.
6	Recruiter	The 15-question evaluation set has been run under both conditions and the comparison is recorded in the README as a single measured result.
7	All	The whole system starts from a single <code>docker compose up</code> on a clean machine.
8	Recruiter	The application is deployed and reachable at a public URL.
9	Examiner	Tests run automatically on every push and pass.
10	All	The README documents the architecture and the design decisions, including the alternatives rejected.
11	Public	A demo video exists and is publishable.
12	All	A user can register, log in, and see only their own query history. Sessions expire.
13	Examiner	The SQL that executed provably joins along the path the trace reports; a mismatch is detected and surfaced rather than passing silently.
14	All	A reader who does not write SQL can tell from the trace what each join path means and which one was taken.
15	Recruiter	The repository contains no credential or API key, in the working tree or anywhere in its history.

10 Position and next step

Phase 1 closes on sign-off of Section 9; Phase 2 begins immediately after.

PHASE	STATUS	NOTE
1 • Planning & feasibility	Closed	This document, at version 2.
2 • Requirements	Closed	99 requirements across four categories. Deliverable: the SRS, version 2.
3 • Design	Closed	Software Design Document v2 — 20 decisions, ordered build sequence, checkpoint ritual.
4 • Build	Next	Fourteen deliverables. Step 1 stands the stack up empty; step 2 loads the warehouse and verifies the graph.
4-7 • Build, test, deploy, maintain	Not started	—

IMMEDIATE NEXT ACTION

Begin the build. Step 1 stands the stack up empty and confirms free-tier hosting before anything depends on it. Step 2 loads TPC-DS, applies `tpcds_ri.sql`, and verifies the resulting graph — including the three routes from a sale to a region that the demonstration rests on.

11 Revision history

What changed, and why, so the reasoning survives alongside the result.

VERSION	DATE	CHANGE
1	5 Sep 2026	Issued. Purpose, scope, feasibility, constraints, four decisions, eight risks, definition of done in eleven criteria.
3	7 Sep 2026	Capacity audit. The eight-day plan was estimated at ~116 hours against 96 available. One day added, first week of October reserved as contingency (D-09), and the auth-versus-deployment sacrifice thereby dissolved (D-11). Verification pass. TPC-DS confirmed at 24 tables with no declared constraints; <code>tpcds_ri.sql</code> adopted as the relationship source (D-10); R-09 downgraded from High to Low; R-10 added. Two errors corrected on the record: the foreign key count was stated as "roughly thirty" and is roughly eighty; and a proposed fix — deriving relationships from the <code>_sk</code> naming convention — was unsound, since about half the columns break it.
2	6 Sep 2026	Definition of done extended to fifteen criteria (12 authentication, 13 trace accuracy, 14 trace readability, 15 no secrets in the repository). Decision log extended with D-05 through D-08, including the authentication reversal. Risk R-09 added following confirmation that TPC-DS declares no foreign keys. Phase status updated. Prompted by a holistic audit before the SRS was finalised; criteria 13 and 14 exist because the original definition permitted a trace that was present, wrong and unreadable.